

Document number: P0237R0

Date: 2016-02-12

Project: ISO JTC1/SC22/WG21: Programming Language C++

Audience: Library Evolution Working Group, SG14

Reply to: Vincent Reverdy (vince.rev@gmail.com)

On the standardization of fundamental bit manipulation utilities

Vincent Reverdy¹ and Robert J. Brunner¹

¹*Department of Astronomy, University of Illinois at Urbana-Champaign, 1002 W. Green St., Urbana, IL 61801*

Abstract

We discuss the addition to the standard library of class templates to ease the manipulation of bits in C++. This includes a `bit_value` class emulating a single bit, a `bit_reference` emulating a reference to a bit, a `bit_pointer` emulating a pointer to a bit, and a `bit_iterator` to iterate on bits. These tools would provide a solid foundation of algorithms operating on bits and would facilitate the use of unsigned integers as bit containers.

Contents

1	Introduction	2
2	Motivation	2
3	Impact on the standard	3
4	Design decisions	4
5	Technical specifications	16
6	Alternative technical specifications	26
7	Discussion and open questions	29
8	Acknowledgements	30
9	References	31

1 Introduction

This proposal introduces a class template `std::bit_reference` that is designed to emulate a reference to a bit. It is inspired by the existing nested classes of the standard library: `std::bitset::reference` and `std::vector<bool>::reference`, but this new class is made available to C++ developers as a basic tool to construct their own bit containers and algorithms. It is supplemented by a `std::bit_value` class to deal with non-referenced and temporary bit values. To provide a complete and consistent set of tools, we also introduce a `std::bit_pointer` in order to emulate the behaviour of a pointer to a bit. Based upon these class templates, we design a `std::bit_iterator` that provides a foundation of bit manipulation algorithms. We discuss the API that is required to access the underlying representation of bits in order to make these algorithms faster. Although they will be given as illustrating examples, bit algorithms would need a separate proposal and are thus considered as out of the scope of this proposal that focuses on the fundamental tools.

2 Motivation

In *The C++ Programming Language* [Stroustrup, 2013], Bjarne Stroustrup highlights the fact that “unsigned integer types are ideal for uses that treat storage as a bit array.” One of the most basic functionality that an array generally provides is a convenient way to access its elements. However, the C++ standard library is currently missing a tool to access single bits in a standardized way. Such tools already exist, but they are buried as internal helper classes with private constructors and thus they are kept away from C++ developers. Specific examples include `std::bitset::reference`, `std::vector<bool>::reference` and `boost::dynamic_bitset::reference` [Siek et al., 2015]. If unsigned integral types should be seen as bit containers, it would be convenient to have a standard utility to access and operate on single bits as if they were array elements.

In addition to this basic motivation, applications that could leverage bit utilities include, among others, performance oriented software development for portable devices, servers, data centers and supercomputers. Making the most of these architectures often involves low-level optimizations and cache-efficient data structures [Carruth, 2014]. In fact, these aspects are going to become more and more critical in a post-Moore era where energy efficiency is a primary concern. In that context, being able to act directly on bits, for example to design efficient data structures based on hash tables, is of primary importance. Moreover, the spread of arbitrary-precision integral arithmetic both at the hardware level [Ozturk et al., 2012] and at the software level, as proposed in N4038 [Becker, 2014], will require, once again, tools to efficiently access single bits.

For all of these reasons, and to prevent bit references to be repeatedly implemented, we propose to add a `std::bit_reference` class template to the C++ standard library. As a response to feedback gathered through the [future proposal](#) platform, we have complemented this class template with a `std::bit_value`, a `std::bit_pointer` and a `std::bit_iterator` in order to have a complete set of bit utilities, and to serve as the basis of a future standardized library of bit algorithms based on an alternative and more generic approach than N3864 [Fioravante, 2014].

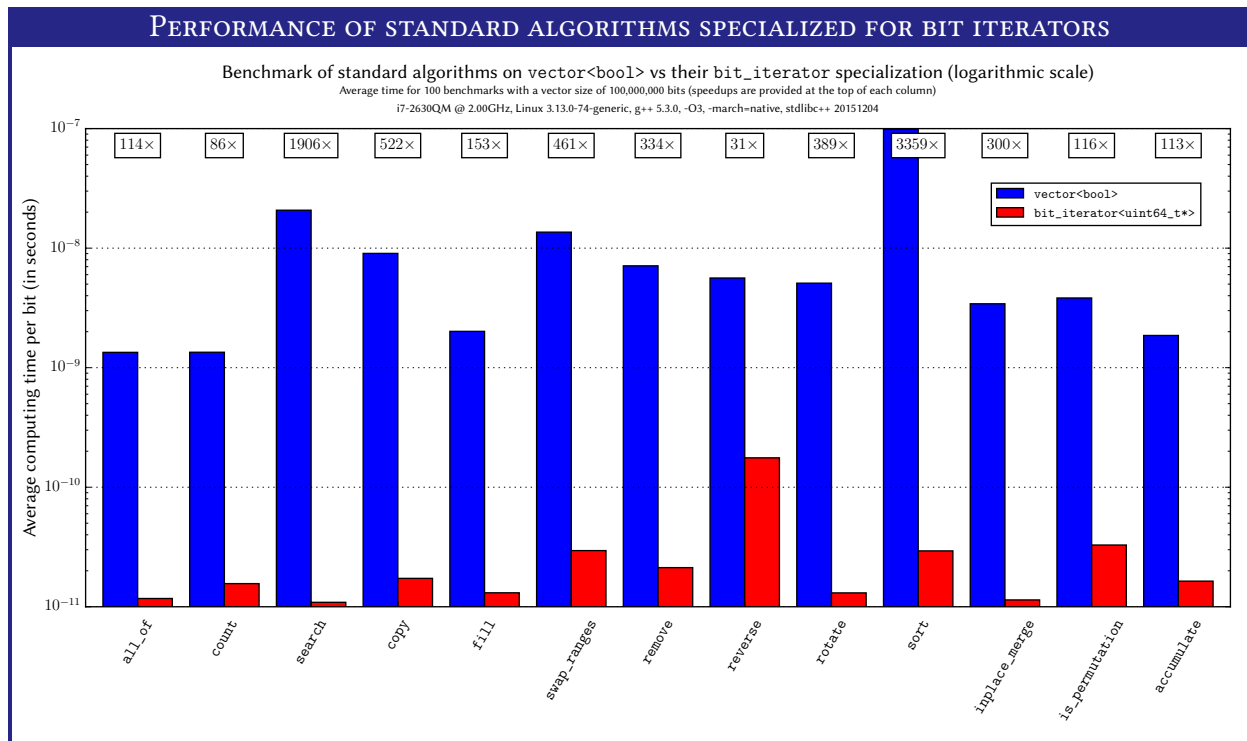


Figure 1: Bit algorithms performances.

With bit iterators, some standard algorithms could benefit from substantial optimizations. For example, a specialization of `std::count` on `std::bit_iterator` should be able to call, when available, the assembly instruction `popcnt` on the underlying unsigned integers of the bit sequence. `std::sort` could also call `popcnt` to count the number of zeroes and ones, and then directly change the value of unsigned integers accordingly. In fact, most standard algorithms, such as `std::copy`, should be able to operate directly on integers instead of individual bits. These types of approaches have already been explored in [libc++](#) for `std::vector<bool>` with [significant performance improvements](#) [Hinnant, 2012]. Specialized bit algorithms could also be provided. As an example, a `parallel_bit_deposit` algorithm could be far more efficient than a `std::copy_if` by calling the assembly function `pdep` on integers. Figure 1 summarizes benchmark results comparing the performance of standard algorithms called on `std::vector<bool>` and their bit iterator counterpart implementing the design described in this proposal. As shown, most algorithms can benefit from speedups of more than two orders of magnitude. A library of bit utilities as described here would allow users to write their own efficient bit algorithms using similar strategies: such utilities would provide a unifying generic zero-overhead abstraction to access CPU intrinsics such as instructions from [Bit Manipulation Instruction sets](#) or from the bit-band and bit manipulation engines on ARM-Cortex architectures [Yangtao, 2013].

3 Impact on the standard

This proposal is a pure library extension. It does not require changes to any standard classes or functions, and introduces a new header for bit utilities whose name is discussed in part 4. Section 7

discusses the nested classes `std::bitset::reference` and `std::vector<bool>::reference`.

4 Design decisions

Introduction

We propose a `<bit>` header providing a class `std::bit_value` and three class templates parameterized by a type: a `std::bit_reference`, with a design inspired by the existing nested bit reference classes [ISO, 2014], a `std::bit_pointer` and a `std::bit_iterator`. The following subsections explore the design parameter space. Even if a lot of attention is given to the design decisions concerning bit values and bit references, the original motivation of this proposal remains `std::bit_iterator` which provides an entry point for generic bit manipulation algorithms. `std::bit_value`, `std::bit_reference`, `std::bit_pointer` are additional classes answering the question: what should `std::bit_iterator::value_type`, `std::bit_iterator::reference` and `std::bit_iterator::pointer` be?

Background

A clear definition of what a bit is, how it is related to bytes and to fundamental types, and what its behaviour should be like are prerequisites of well designed bit utility classes. The need of raising the question of the definition of a bit can be illustrated by the following problem, where 0 and 1 indicate the bit value obtained at the end of the line, and where X refers to non-compiling lines:

```
1 struct field {unsigned int b : 1;};
2
3 bool b0 = false; b0 = ~b0; b0 = ~b0; // 1
4 auto x0 = std::bitset<1>{}[0]; x0 = ~x0; x0 = ~x0; // 0
5 auto f0 = field{}; f0.b = ~f0.b; f0.b = ~f0.b; // 0
6
7 bool b1 = false; b1 = ~b1; // 0
8 auto x1 = std::bitset<1>{}[0]; x1 = ~x1; // 1
9 auto f1 = field{}; f1.b = ~f1.b; // 0
10
11 bool b2 = false; b2 += 1; b2 += 1; // 1
12 auto x2 = std::bitset<1>{}[0]; x2 += 1; x2 += 1; // X
13 auto f2 = field{}; f2.b += 1; f2.b += 1; // 0
14
15 bool b3 = false; b2 = b3 + 1; b3 = b3 + 1; // 1
16 auto x3 = std::bitset<1>{}[0]; x3 = x3 + 1; x3 = x3 + 1; // 1
17 auto f3 = field{}; f3.b = f3.b + 1; f3.b = f3.b + 1; // 0
18
19 bool b4 = false; b4 += 3; // 1
20 auto x4 = std::bitset<1>{}[0]; x4 += 3; // X
21 auto f4 = field{}; f4.b += 3; // 1
```

As shown in this example, three existing C++ bit-like entities exhibit three different behaviours. Given that `std::bit_value` and `std::bit_reference` will define an arithmetic behaviour for a bit, it is important to think carefully about what this behaviour should be. Also, before discussing in more details the chosen design and its alternatives, we summarize what the existing

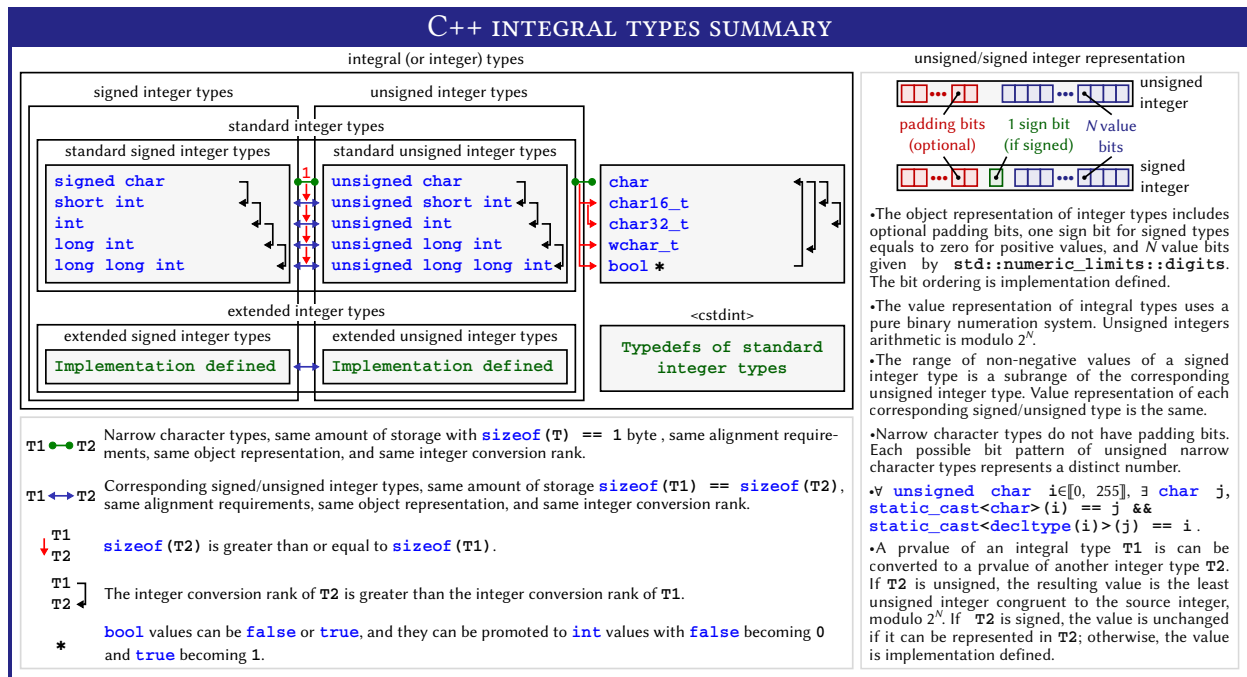


Figure 2: Integral types.

standards have to say on bits and bytes, as well as on integral types, using the C++ working draft N4567 [Smith, 2015] and the C working draft N1548 [Jones, 2011]. The purpose of the following paragraphs is to provide condensed background information from the standards related to this proposal before starting discussing the design decisions in the next subsection.

The C standard gives the following definition in its section 3.5: a *bit* is a unit of data storage in the execution environment large enough to hold an object that may have one of two values. The C++ standard defines a bit in [intro.memory] as an element of a contiguous sequence forming a byte, a *byte* being the fundamental storage unit in the C++ memory model. According to this model, the memory available to a C++ program consists of one or more sequences of contiguous bytes. A byte is required to have a unique address and to be at least large enough to contain any member of the basic execution character set and the eight-bit code units of the Unicode UTF-8 encoding form. An *object* is defined in [intro.object] as a region of storage. According to [intro.memory] the address of an object is the address of the first byte it occupies, unless this object is a bit-field or a base class subobject of zero size. The section [basic.types] defines two representations. The *object representation* of an object of type T is the sequence of N `unsigned char` objects taken up by the object of type T . The number N is given by `sizeof`, where the `sizeof` operator yields the number of bytes in the object representation of its operand according to [expr.sizeof]. In the C++ standard, [basic.types] states that for any object, other than a base-class subobject, of trivially copyable type T , whether or not the object holds a valid value of type T , the underlying bytes making up the object can be copied into an array of `char` or `unsigned char`. The relationship between bytes and characters is made clearer in [basic.fundamental] and [expr.sizeof], as well as in the section 6.2.6.1 of the C standard which establishes a direct link between bytes and `unsigned char`. The other representation defined by the C++ standard is the *value representation*. It corresponds to the set of bits that hold the value of type T .

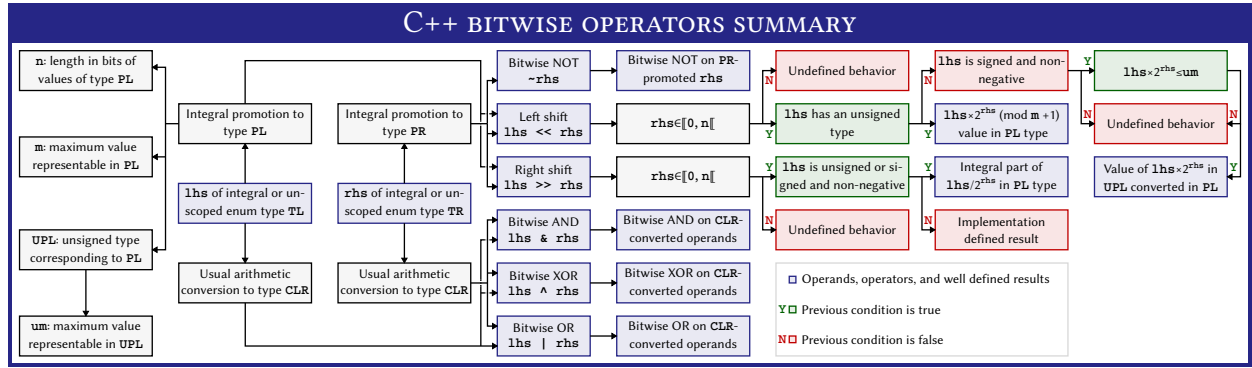


Figure 3: Bitwise operators.

Regarding integers, [fundamental.types] defines five *standard signed integer types*, five *standard unsigned integer types* and additional *extended integer types*. Figure 2 summarizes the properties of the C++ *integral types*, their representation and their conversion rules according to [fundamental.types], [cstdint.syn], [numeric.limits.members], [conv.prom], [conv.integral] and [conv.rank]. As stated in this figure, the C++ standard require representation of integral types to define values by use of a pure binary numeration system. Such a system corresponds to a positional representation of integers that uses the binary digits 0 and 1, in which the values represented by successive bits are additive, begin with 1, and are multiplied by successive integral power of 2, except perhaps for the bit with the highest position [ISO, 2014]. Integral types come with bitwise operators whose behaviour is presented on figure 3 and that are of primary interest regarding the topic of this proposal for the role they play in bit extraction. In top of integral types, booleans, and character types, fundamental types also include floating point types. A synoptic view of conversion rules involved in arithmetic operations for all these types is given on figure 4 for an implementation compliant with the C++ standard. In this figure, the type of x corresponds to rows while the type of y corresponds to columns. Each type is associated with a color that is used to indicate the decayed result type of an operation involving x and y . As an example, for x a `long long int` and for y an `unsigned long int`, the type of $x + y$ is `unsigned long long int`. An interesting property to note is that integral types smaller than `int` are implicitly converted to `int` during arithmetic operations regardless of their signedness: as an example, a `bool`, a `char`, an `unsigned char` and an `unsigned short int` exhibit similar arithmetic behaviours for most operations.

In this context, two questions regarding the definition and the behaviour of a bit appear, and are at the core of the design of the class templates we propose:

- How to define the position of a bit within an object?
- What is the arithmetic definition of a bit?

The consequences of the first question include the types on which `std::bit_reference` and `std::bit_pointer` will operate, and what bits will be accessible through iteration. The answer to the second question will determine the implicit conversions and the results of arithmetic operations on `std::bit_value` and `std::bit_reference`.

RESULT TYPE OF ARITHMETIC OPERATIONS ON C++ FUNDAMENTAL TYPES

	unary operators							
	++x	--x	*++	*--	++*	--*	++*	
bool								
char								
char16_t								
char32_t								
wchar_t								
signed char								
unsigned char								
short int								
unsigned short int								
int								
unsigned int								
long int								
unsigned long int								
long long int								
unsigned long long int								
float								
double								
long double								

	x + y							
	bool	char	char16_t	char32_t	wchar_t	signed char	short int	
bool								
char								
char16_t								
char32_t								
wchar_t								
signed char								
unsigned char								
short int								
unsigned short int								
int								
unsigned int								
long int								
unsigned long int								
long long int								
unsigned long long int								
float								
double								
long double								

	x * y							
	bool	char	char16_t	char32_t	wchar_t	signed char	short int	
bool								
char								
char16_t								
char32_t								
wchar_t								
signed char								
unsigned char								
short int								
unsigned short int								
int								
unsigned int								
long int								
unsigned long int								
long long int								
unsigned long long int								
float								
double								
long double								

	x / y							
	bool	char	char16_t	char32_t	wchar_t	signed char	short int	
bool								
char								
char16_t								
char32_t								
wchar_t								
signed char								
unsigned char								
short int								
unsigned short int								
int								
unsigned int								
long int								
unsigned long int								
long long int								
unsigned long long int								
float								
double								
long double								

	x & y							
	bool	char	char16_t	char32_t	wchar_t	signed char	short int	
bool								
char								
char16_t								
char32_t								
wchar_t								
signed char								
unsigned char								
short int								
unsigned short int								
int								
unsigned int								
long int								
unsigned long int								
long long int								
unsigned long long int								
float								
double								
long double								

	x &= y							
	bool	char	char16_t	char32_t	wchar_t	signed char	short int	
bool								
char								
char16_t								
char32_t								
wchar_t								
signed char								
unsigned char								
short int								
unsigned short int								
int								
unsigned int								
long int								
unsigned long int								
long long int								
unsigned long long int								
float								
double								
long double								

	x &= y							
	bool	char	char16_t	char32_t	wchar_t	signed char	short int	
bool								
char								
char16_t								
char32_t								
wchar_t								
signed char								
unsigned char								
short int								
unsigned short int								
int								
unsigned int								
long int								
unsigned long int								
long long int								
unsigned long long int								
float								
double								
long double								

	x &= y							
	bool	char	char16_t	char32_t	wchar_t	signed char	short int	
bool								
char								
char16_t								
char32_t								
wchar_t								
signed char								
unsigned char								
short int								
unsigned short int								
int								
unsigned int								
long int								
unsigned long int								
long long int								
unsigned long long int								
float								
double								
long double								

	x &= y							
	bool	char	char16_t	char32_t	wchar_t	signed char	short int	
bool								
char								
char16_t								
char32_t								
wchar_t								
signed char								
unsigned char								
short int								
unsigned short int								
int								
unsigned int								
long int								
unsigned long int								
long long int								
unsigned long long int								
float								
double								
long double								

	x &= y							
	bool	char	char16_t	char32_t	wchar_t	signed char	short int	
bool								
char								
char16_t								
char32_t								
wchar_t								
signed char								
unsigned char								
short int								
unsigned short int								
int								
unsigned int								
long int								
unsigned long int								
long long int								
unsigned long long int								
float								
double								
long double								

Figure 4: Arithmetic operations on fundamental types.

How to define the position of a bit within an object?

Bits are not directly addressable, but they are defined as binary elements of bytes which are the most fundamental addressable entities of a given system and are required to be made of at least 8 bits. Consequently, identifying a bit requires a byte *address* and a *position* within a byte. The problem is that the underlying ordering of bits within a byte is not specified by the standard. Therefore, according to the sole criterion of bits seen as elements of bytes, the mapping between a position and an actual bit is implementation-defined.

To make bit references, pointers, and iterators usable, the design needs to specify this mapping. As presented in the [background](#) subsection, the standard defines a clear connection between bytes and [unsigned chars](#): an [unsigned char](#) have a size of exactly one byte, has no padding or sign bits, each possible bit pattern represents a distinct number, and its value rely on a pure binary numeration system. In other words, [unsigned chars](#) define an unambiguous bit mapping which corresponds to the definition of a bit seen as a *binary digit* of natural numbers. According to this mapping, the n -th bit of an [unsigned char](#) `uc` is obtained by the operation `uc >> n & 1`, when $n \in \llbracket 0, \text{std::numeric_limits}<\text{unsigned char}>::\text{digits} \rrbracket$. With this definition, every bit can be referenced in an univocal manner with a pair of byte address and position `std::pair<unsigned char*, std::size_t>`.

Although it is very well defined, this method is very limited in the sense that it only gives access to the object representation of types, and does not provide a direct implementation independent way of accessing the n -th bit of the value representation of integral types. In fact, as the object representation of integers other than the unsigned narrow character types is implementation-defined, the method described above gives access to all bits of the integers, but in an order that can depends on the architecture and on the compiler. Endianness and padding bits are, of course, a part of the problem. Defining a bit as the n -th binary digit of a natural number makes the design more generic and more usable. According to this definition, for any unsigned integer `ui` of type `UIntType`, we can obtain the n -th bit by the same formula we used for the [unsigned char](#) case: `ui >> n & 1` for $n \in \llbracket 0, \text{std::numeric_limits}<\text{UIntType}>::\text{digits} \rrbracket$. A design relying on this approach presents several advantages: it defines unambiguously the position of a bit for all unsigned integer types, it produces a platform-independent behaviour regardless of the underlying representation of these integers, their endianness and the number of padding bits they include, and it still provides an access to the object representation through a [reinterpret_cast](#) to [unsigned chars](#). Additionally, and more importantly, the definition of the position of a bit matches its mathematical definition in a *positional numeration system*, making the use of the design intuitive.

At this point, the question of the generalization of this design arises. Should types other than unsigned integer types be allowed? For a complete arbitrary type `T`, the only relevant bit definition is the one based upon the object representation of `T`. The design proposed in the previous paragraph can already provides an easy access to the object representation of `T` through a [reinterpret_cast](#) to [unsigned char](#) pointers. However, the question remains open for the following types: non-integral arithmetic types, bit containers, unbounded-precision integer types as proposed in [N4038](#) [[Becker, 2014](#)] and, of course, non-unsigned integral types. Concerning floating-point types, as their underlying representation is implementation-defined as specified in [basic.fundamental] and is left completely free by the standard, as this representation is not relying on a pure positional numeration system but generally includes a sign, a mantissa and an exponent, and as the shift operators does not apply to them, it does not make much sense to treat them differently than any other arbitrary type `T`. For the three remaining cases, namely bit containers, unbounded-precision integer types and non-unsigned integral types, the situation is different since one can define a bit position relying on the value representation of these objects.

The question of referencing a bit in bit containers, like `std::bitset` and the specialization

`std::vector<bool>`, and in unbounded-precision integer types is very similar. Even if not required, the vast majority of implementations of these objects rely on contiguous arrays of *limbs* of unsigned integer types. For bit containers, the most natural definition of the bit position would be the same as the one entering in the declaration of the subscript operator `operator[]`. For unbounded-precision integers it can be trickier since they can be signed and include a sign bit. But even if we ignore, for the moment, the issue of the sign bit, other design questions exist. For example, it is unlikely that most unbounded-precision integers define a subscript operator. In this case, accessing a bit through the shift operator, as in the previous paragraphs, would make more sense. This technique could also apply to `std::bitset` but not to `std::vector<bool>` since the specialization does not provide an `operator>>`. Moreover if a maker helper function such as `make_bit_reference(T& object, std::size_t pos)` was provided for bit containers and unbounded-precision integers, should the bit reference behaviour rely on the object, or on its underlying representation in terms of limbs? Regarding to this question, it would make more sense to rely on the object and its `operator[]` or `operator>>` regardless of the underlying representation in the similar way a bit reference relying on unsigned integers would work regardless of the optional presence padding bits. An internal access to the underlying container of limbs could still be provided through member functions returning `std::bit_references` instead of `std::bit_references` taking bit containers as parameters. It also opens the question of whether or not `std::bitset::reference` and `std::vector<bool>::reference` should be replaced by a `std::bit_reference`, or at least adjusted to provide the same interface. As already noted, for unbounded-precision or non-unsigned integer types, the question of the sign bit and negative values also has to be solved. In fact, the mapping between the object representation of signed integers and their negative values is far less constrained than for unsigned integer types. Consequently, a design relying on `operator>>` would lead to implementation-defined results. Whether we should, or not, accept such a design is left as an open question. As a remark, the feedback gathered online from the [C++ standard discussion board](#) pointed out that bit manipulation on signed integers could be achieved with a design limited to unsigned integer types, through a `reinterpret_cast<typename std::make_unsigned<IntType>::type*>`.

In all the following, we restrain the design to unsigned integer types as defined on figure 2. We also define the bit position such as the expression `ui >> n & 1` is extracting the *n*-th binary digit for $n \in [0, \text{std::numeric_limits}<T>::\text{digits}]$. This choice is motivated by the fact that:

- the bit position matches the mathematical definition of a binary digit position in a positional numeration system
- it leads to a platform-independent behaviour
- it provides an access to the underlying bits of any type through a `reinterpret_cast` to `unsigned char*`
- it provides an access to the bits of signed integer types through a `reinterpret_cast` to `typename std::make_unsigned<IntType>::type*`
- “unsigned integer types are ideal for uses that treat storage as a bit array” as highlighted in section 2
- it matches the requirements of most use cases including cryptographic operations, hash value calculations and computations on arrays of limbs

- classes such as `std::bitset` already set a preference of conversions from and to unsigned integer types over generic integer types

However, and as a final note, the proposed design could stay the same and still accept all integral types, including future unbounded-precision integral types with minor modifications, at the expense of implementation-defined results since its specification is relying on the use of bitwise operations to extract bits.

What is the arithmetic definition of a bit?

The second main question on which a significant part of the design of a bit reference relies concerns the arithmetic behaviour of a bit. As shown in the introductory listing of the [background](#) subsection, three bit-like objects already present in the standard exhibit three different behaviours. In this part, we discuss the different options, their advantages and their drawbacks.

The first option is the one followed by `std::bitset::reference` and `std::vector<bool>::reference`. These classes are nested classes, mostly intended to take care of the result of the subscript operator `operator[]` and implementing the behaviour of a boolean value from the user's point of view. As the goal of `std::bit_value`, `std::bit_reference`, `std::bit_pointer` and `std::bit_iterator` will be slightly different in the sense that they are specifically intended to provide users with the ability of writing their own bit manipulation algorithms, the choices made in terms of arithmetic can be different from the the ones of the nested classes, especially if it leads to a better interface for users. Many approaches tend to identify a bit with a boolean although the two are conceptually different: the first one is a digit whereas the second one is a logical data type. Both happen to have two possible values which generally leads to representation of the first one in terms of the second one. The arithmetic behaviour of `std::bitset::reference` and `std::vector<bool>::reference` mainly relies on the implicit conversion to `bool`. As a consequence, all binary operators applicable to `bool` also applies to `std::bitset::reference` through this implicit conversion. However, the reference does not exactly behave as a `bool` since it provides a `flip` member, it implements its own `operator~`, and it does not allow arithmetic assignment operations. In other words, if `ref` is of type `std::bitset::reference`, `~ref` can lead to different values than if it were a `bool`, `ref = ref + 3` gives the same result as if it were a `bool`, and `ref += 3` does not compile. For a nested class whose main role is to serve as proxy for the result of `operator[]`, this very specific behaviour may not be of primary concern. But for a `std::bit_reference` designed to provide a generic way to deal with bit operations, an implicit conversion to `bool` and its implicit integral promotion to `int` mixed with specifically designed operators such as `operator~` could be very error-prone. Also, we investigate other alternatives to the original scenario which would consists in reproducing the exact same behaviour as `std::bitset::reference`.

The first alternative is to consider that a bit, as a pure binary digit, is not an arithmetic object and therefore should not implement any arithmetic behaviour. Instead, it would provide three member functions: `set`, `reset` and `flip`, these functions already being a part of the implementation details of some bit references such as `boost::dynamic_bitset::reference` [Siek et al., 2015]. Boolean conversions would be provided for arithmetic purpose through an `operator=` and through an *explicit* `operator bool`. The explicitness of the operator would pre-

vent any undesired conversion and integral promotion, and would make clearer the conceptual difference between a binary digit and a boolean data type while still providing the desired casting functionality. This would lead to a minimal but very consistent design.

The second alternative is to consider that a bit and a `bool` have the exact same behaviour. In that case, the class would not provide special members like `set`, `reset` and `flip` and would stick to the arithmetic operators executable on booleans. The binary arithmetic operators could be provided either explicitly, or by an implicit cast to `operator bool`. `operator~` and arithmetic assignment operators would be provided and would lead to the same results as for booleans. As in the case of the first alternative, this strategy would avoid unexpected arithmetic behaviours for users and would introduce an easily understandable interface.

The third alternative echoes the fact that the C++ standard identifies bytes with an unsigned integer type, namely `unsigned chars`. In the same manner, we can consider a bit as a binary digit with an arithmetic behaviour equivalent to a hypothetical `uint1_t`, an unsigned integer one-digit long that can be equal to either $0 \times 2^0 = 0$ or $1 \times 2^0 = 1$. For the binary digit side, `std::bit_reference` would get the member functions `set`, `reset` and `flip` and an explicit `operator bool`. For the arithmetic side, assignment operators and increment and decrement operators would implement a modulo $2^1 = 2$ arithmetic. Consequently, for a bit reference `bit` initially equals to 0, `bit += 3` would lead to a value of 1 and `(++bit)++` would lead to a value of 0. For binary arithmetic operators there are two options to consider: either implementing the overloads explicitly, or making the operators work through an implicit cast to an unsigned integral type. For this last option there are three possibilities: this type could be set to the smallest unsigned integral type, namely `unsigned char` or `uint_least8_t`, or it could be set to the type in which the bit is referenced, or it could be set through an additional template parameter of `std::bit_reference`. However, adding a template parameter to specify the arithmetic behaviour of a bit would made the bit classes more complex for no real benefit.

To summarize, the main possibilities in terms of the arithmetic behaviour of a bit are the following:

- the design of the nested classes `std::bitset::reference` and `std::vector<bool>::reference`, with a mix of behaviours, possibly error-prone
- the first alternative, consisting in considering a bit as a pure binary digit therefore stripped of an arithmetic behaviour, although still accessible through an explicit conversion to a `bool`
- the second alternative, consisting in considering a bit as a boolean and therefore providing the exact same functionalities as a `bool`
- the third alternative, consisting in considering the first alternative with additional arithmetic properties corresponding to a one-digit long unsigned integer

Earliest drafts of this proposal were limited to these four options and the chosen design was based on the first alternative to keep the technical specifications as simple as possible. However, this simplicity was coming with a minor open problem. Considering that `std::bit_iterator::reference` is a `std::bit_reference`, and that `std::bit_iterator::pointer` is a `std::bit_pointer`, then what should `std::bit_iterator::value` be? Defining it as a `bool` or as an `unsigned char` would not provide the arithmetic behaviour of a one-digit long unsigned in-

teger, while defining it as a `std::bit_reference` could lead to errors, since a reference and a value are two different things. Moreover, if `std::bit_reference` implements a one-digit long unsigned integer arithmetic, then what should be returned by the postfix increment and decrement operators? For consistency it has to return a type with the same functionalities as `std::bit_reference`, including `set`, `reset` and `flip` functions, but it cannot be a referenced bit: it has to be an independent bit.

This is where the idea of `std::bit_value` comes into play, solving these problems, allowing a consistent arithmetic behaviour implementation, and simplifying the design of the class template `std::bit_reference`. The role of `std::bit_value` is to mimic the value of independent, non-referenced bits. As a class representing independent bits implicitly constructible from bit references, it has to provide the arithmetic behaviour of a one-digit long unsigned integer. But the question of how to implement these arithmetic operators in a lightweight manner still remains. The answer can be found by analyzing the content of figure 4. The important thing to notice is that, for most operations, `bool`, `unsigned char` and `unsigned short int` act in the same way: they are implicitly casted to `int`, and so should a one-digit long unsigned integer. Fitting `std::bit_value` with an implicit `operator bool` would enable this behaviour. However, making `std::bit_value` implicitly constructible from `bool` would not result in a one-digit long unsigned integer arithmetic, but making it implicitly constructible from `unsigned char` would do it. This strategy leads to a conversion and optionally a narrowing of any integer type to `unsigned char`, whose least significant bit could then be extracted to set the actual value of `std::bit_value`. In addition to these implicit conversions, `std::bit_value` and `std::bit_reference` would have to implement the arithmetic operators that mutate their states such as compound assignment operators and both increment and decrement operators.

This approach, namely:

- the fourth alternative, consisting in considering the third alternative implemented with an additional `std::bit_value` class

is the one that is followed in this proposal because of the solution it provides to the above-mentioned problems.

What design?

Based on the answers to the fundamental questions of the [position of a bit](#) and of its [arithmetic behaviour](#), we can design a library solution to access bits. Accordingly to the previous subsections, this design is built around four elements:

- `std::bit_value` emulating an independent, non-referenced bit
- `std::bit_reference` emulating a reference to a bit
- `std::bit_pointer` emulating a pointer to a bit
- `std::bit_iterator`, based on the preceding classes and emulating an iterator on bits

`std::bit_reference` and `std::bit_pointer` are parameterized by a template type indicating the underlying object type of which the bits belongs to. They both have a `constexpr` constructor taking either a reference or a pointer to a value of the underlying type, and a position indicating the position of the bit. `std::bit_reference` gives access to a `std::bit_pointer`

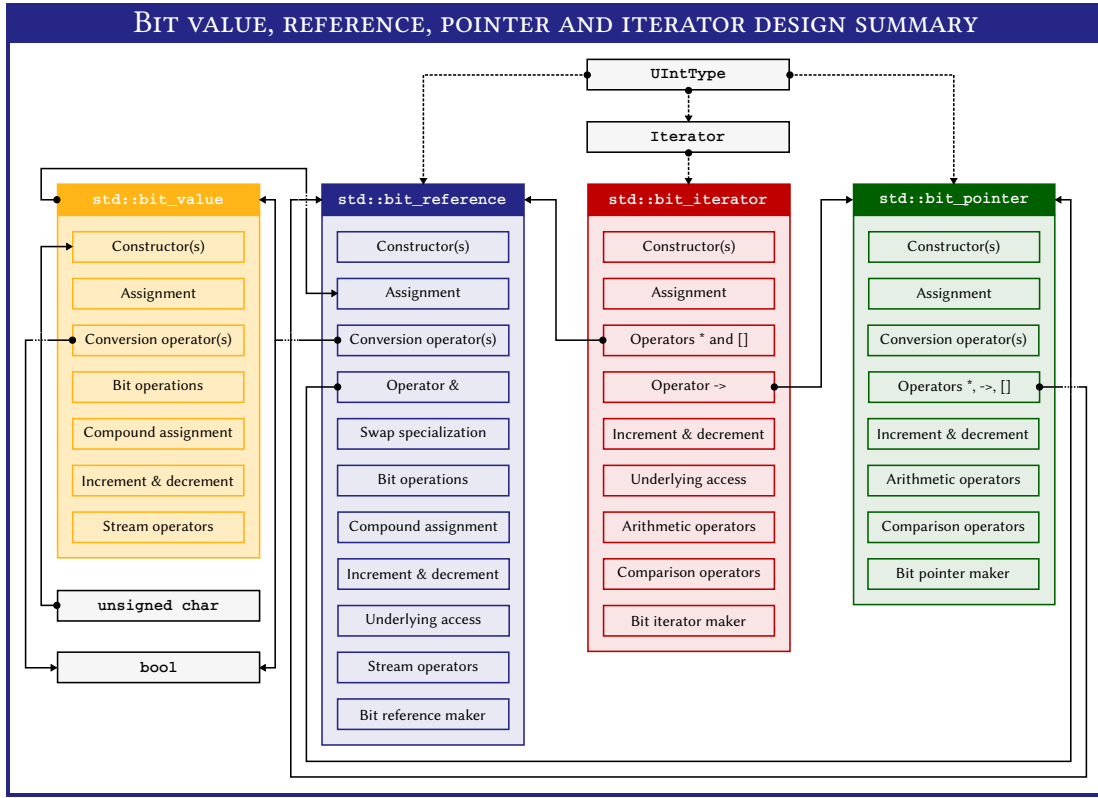


Figure 5: Design summary.

through its member `operator&`, and, reciprocally, `std::bit_pointer` gives access to a bit reference through its operators `operator*`, `operator->` and `operator[]`. `std::bit_reference` implements the behaviour of a bit: it provides basic bit functionalities as well as a conversion operator to a `std::bit_value` and an assignment operator taking a `std::bit_value` as a parameter. `std::bit_value` implements an independent bit and provides the same functionalities as bit references. Stream operators are also overloaded for both bit values and references, to provide a display of 0 and 1 values. Finally, an interface to access the underlying information of `std::bit_reference`, namely the address of the referenced object and the bit position, is provided to allow the writing of faster bit manipulation algorithms. `std::bit_pointer` emulates the behaviour of a pointer to a bit, implementing all the classical functions operating on traditional pointers. A `std::bit_pointer` can be nullified, and in that case, the underlying pointer is set to `nullptr` and the position is set to 0. `std::bit_iterator` is built on the top of both `std::bit_reference` and `std::bit_pointer`. It takes an iterator `Iterator` on an underlying object type as a template parameter. `std::bit_iterator::operator++` and other increment and decrement operators implement the following behaviour: they iterate through the binary digits of the underlying object, and execute the member `Iterator::operator++` to go to the next object once the last binary digit of the current object has been reached. This strategy allows to iterate through contiguous, reversed, non-contiguous and virtually all possible iterable sequences of unsigned integers. A `std::bit_iterator` can be constructed from an `Iterator` value and a position, and it implements the traditional behaviour of a standard iterator, with its value type being a `std::bit_value`, its reference type being a `std::bit_reference`, its pointer type being a `std::bit_pointer` and its category being `std::iterator_traits<Iterator>::iterator_`

category. Finally, and for convenience, the classes come with non-member functions to make the right type of `std::bit_reference`, `std::bit_pointer` or `std::bit_iterator` based on a provided reference, pointer, or iterator. All the design decisions are summarized in figure 5.

Additional remarks: implicit conversions, swap operations, and cv-qualifiers

Additionally to the main design decisions listed in the [previous](#) subsection, some details deserve a particular attention. The first one concern the implicit conversions between bit values and bit references. A straightforward approach would be limited to the following:

- `std::bit_value` is implicitly constructible from `unsigned char`
- `std::bit_value` is implicitly convertible to `bool`
- `std::bit_reference` is assignable from `std::bit_value`
- `std::bit_reference` is implicitly convertible to `std::bit_value`

The problem with this strategy is that a bit reference would be two implicit conversions away from binary arithmetic operators: in other words, adding a bit reference to another arithmetic type would need a first conversion to `std::bit_value` and a second conversion to `bool`. But these two conversions are *user-defined* and the [class.conv] section of the standard specifies that, at most, one user-defined conversion can be implicitly applied to a single value. Consequently, and to avoid this problem, `std::bit_reference` should be made implicitly convertible to `bool`. However, whether or not bit references should remain implicitly convertible to `std::bit_value` too, as in the proposed design, is an open question.

The second remark concerns the `std::swap` function. Because the copy constructor and the copy assignment operator of `std::bit_reference` do not act in the same way, `std::swap` has to be overloaded. If we consider that bit values and bit references model the same fundamental concept of a bit, we should consider the following overloads:

```
1 template <class UIntType>
2 void std::swap(
3     std::bit_value& x,
4     std::bit_reference<UIntType> y
5 );
6 template <class UIntType>
7 void std::swap(
8     std::bit_reference<UIntType> x,
9     std::bit_value& y
10 );
11 template <class UIntType1, class UIntType2>
12 void std::swap(
13     std::bit_reference<UIntType1> x,
14     std::bit_reference<UIntType2> y
15 );
```

Additionally, we should consider overloading the `std::exchange` function because the generic version will not lead to the expected result for bit references:

```
1 template <class UIntType, class U = std::bit_value>
2 std::bit_value std::exchange(
3     std::bit_reference<UIntType> obj,
4     U&& new_val
```

These overloads of `std::swap` and `std::exchange` are not currently included in the design, but are left for discussion. Note that the same kind of questions arise for the comparison operators of bit pointers.

The third remark involves cv-qualified bit references and pointers. If we consider a hypothetical user-defined bit container, how should the typedefs `const_reference` and `const_pointer` be defined? For clarity, we list below all the possibilities regarding the constness of bit references and pointers, with `T` being a non cv-qualified unsigned integer type and with `bit` being a hypothetical fundamental arithmetic type representing a bit:

- `std::bit_reference<T>` models a standard non cv-qualified reference, which is equivalent to a `bit&`
- `std::bit_reference<const T>` models a reference to a constant and therefore mimics a `const bit&`
- `const std::bit_reference<T>` models a constant reference to a non-constant type and is the theoretical equivalent of a hypothetical `bit& const`, which does not compile
- `const std::bit_reference<const T>` models a constant reference to a constant type and is the theoretical equivalent of a hypothetical `const bit& const`, which does not compile
- `std::bit_pointer<T>` models a standard non cv-qualified pointer, equivalent to `bit*`
- `std::bit_pointer<const T>` models a pointer to a constant and mimics a `const bit*`
- `const std::bit_pointer<T>` models a constant pointer to a non-constant type and therefore mimics a `bit* const`
- `const std::bit_pointer<const T>` models a constant pointer to a constant type and therefore mimics a `const bit* const`

Consequently, even if both const-qualified types `const std::bit_reference<const T>` and `const std::bit_reference<T>` compile and can be useful as proxies to carry information about the location and the position of a referenced bit, they should be used with care as they do not have non-proxy equivalents. Moreover, given the listed definitions, it appears more clearly that the `operator->` of `const std::bit_pointer<T>` should return a pointer to a non cv-qualified bit reference, or, in other words, a `std::bit_reference<T>*`, instead of a `const std::bit_reference<T>*`. And to answer the original question, a `const_reference` typedef should be defined as a `std::bit_reference<const T>` and a `const_pointer` typedef as a `std::bit_pointer<const T>`. The last remark concern the implicit cv conversions of bit references and bit pointers. In both cases, a default copy constructor and a constructor taking a reference to the provided template parameter type as an input already handle most cases. However, a `std::bit_reference<const T>` cannot be constructed from a `std::bit_reference<T>`, and a `std::bit_pointer<const T>` cannot be constructed from a `std::bit_pointer<T>`. To make it possible, we have to add generic conversion constructors of the form `template <class T> bit_reference(const bit_reference<T>& other)` and of the form `template <class T> bit_pointer(const bit_pointer<T>& other)`. For bit pointers, an additional generic conversion assignment operator is also required. This last point conclude the remarks and allow us to detail the technical specifications.

5 Technical specifications

Introduction

The design decisions described in section 4, lead to the technical specifications presented in the following pages. A working C++14 implementation will be made available on a public [GitHub](#) repository [Reverdy, 2016].

Naming

Before discussing the definitions of the bit utility class templates, we list all the names related to this proposal, as well as possible alternatives. When these names already exist in the standard library, or are inspired by existing names, they appear in blue and we provide the link of their original source. Parentheses are used for optional prefixes and suffixes and to avoid listing all possible combinations. We start with the header name associated with the classes of this proposal and which could be extended through for future work on bits:

NAMING SUMMARY: HEADER		
Description	Name	Alternatives
Header (bit utilities, bit manipulation functions...)	<code><bit></code>	<code><bits></code> <code><bitwise></code> <code><bit_utility></code> <code><bitutils></code> <code><bit_tools></code>

Then, we list the main class names. We prefer `std::bit_value` over `std::bit` because the second one could be misleading, since the class it refers to does not correspond to a single bit in memory, but instead wraps the value of a bit and provides the desired functionalities.

NAMING SUMMARY: CLASSES		
Description	Name	Alternatives
Bit value class	<code>bit_value</code>	<code>bit</code> <code>bitval</code> <code>bit_val</code>
Bit reference class template	<code>bit_reference</code>	<code>bitref</code> <code>bit_ref</code>
Bit pointer class template	<code>bit_pointer</code>	<code>bitptr</code> <code>bit_ptr</code>
Bit iterator class template	<code>bit_iterator</code>	<code>bititer</code> <code>bit_iter</code>

Then, we list the names used for template parameters:

NAMING SUMMARY: TEMPLATE PARAMETERS		
Description	Name	Alternatives
Generic type	<code>T</code>	<code>Type</code>
Other generic type	<code>U</code>	<code>Other<Type></code>
Unsigned integer type	<code>UIntType</code>	<code>UInt</code> <code>UnsignedInteger<Type></code>
Iterator type	<code>Iterator</code>	<code>It</code>
Character type	<code>CharT</code>	
Character traits type	<code>Traits</code>	

and the names of member typedefs:

NAMING SUMMARY: MEMBER TYPES		
Description	Name	Alternatives
Byte type from which a bit value is constructible	<code>byte_type</code>	<code>byte</code> <code>byte_t</code>
Type to which a bit belongs to	<code>underlying_type</code>	<code>object_type</code> <code>element_type</code> <code>storage_type</code>
Bit position type	<code>size_type</code>	<code>position_type</code> <code>shift_type</code> <code>offset_type</code>
Bit distance type	<code>difference_type</code>	
Base iterator type	<code>iterator_type</code>	<code>underlying_iterator<_type></code>
Iterator traits member types	<code>value_type</code> <code>difference_type</code> <code>pointer</code> <code>reference</code> <code>iterator_category</code>	

Then, we list the names of function members:

NAMING SUMMARY: FUNCTION MEMBERS		
Description	Name	Alternatives
Swap function member	<code>swap</code>	
Set bit function member	<code>set</code>	
Reset bit function member	<code>reset</code>	
Flip bit function member	<code>flip</code>	
Underlying iterator access function member	<code>base</code>	<code>(get_)(underlying_)iterator</code>
Bit memory address access function member	<code>address</code>	<code>addressof</code> <code>(get_)(underlying_)address</code> <code>(get_)(underlying_)pointer</code> <code>(get_)(underlying_)ptr</code>
Bit position access function member	<code>position</code>	<code>(get_)(underlying_)position</code> <code>(get_)(underlying_)pos</code> <code>(get_)(underlying_)shift</code> <code>(get_)(underlying_)offset</code>

as well as the names of non-member functions:

NAMING SUMMARY: FUNCTIONS		
Description	Name	Alternatives
Non-member swap function	<code>swap</code>	
Bit reference creation function	<code>make_bit_reference</code>	<code>make_bitref</code> <code>make_bit_ref</code>
Bit pointer creation function	<code>make_bit_pointer</code>	<code>make_bitptr</code> <code>make_bit_ptr</code>
Bit iterator creation function	<code>make_bit_iterator</code>	<code>make_bititer</code> <code>make_bit_iter</code>

A finally, the following names are used for function parameters:

NAMING SUMMARY: PARAMETERS		
Description	Name	Alternatives
Reference, pointer and iterator	<code>ref</code> <code>ptr</code> <code>i</code>	
Position	<code>pos</code>	
Value to be assigned	<code>val</code>	
Increment or decrement	<code>n</code>	
Object to be copied or assigned	<code>other</code>	
Left-hand and right-hand sides of an operator	<code>lhs</code> <code>rhs</code>	
Output and input streams	<code>os</code> <code>is</code>	
Bit reference, pointer or iterator in non-member functions	<code>x</code>	

Bit value specifications

The specifications of `std::bit_value` are given on figure 6.

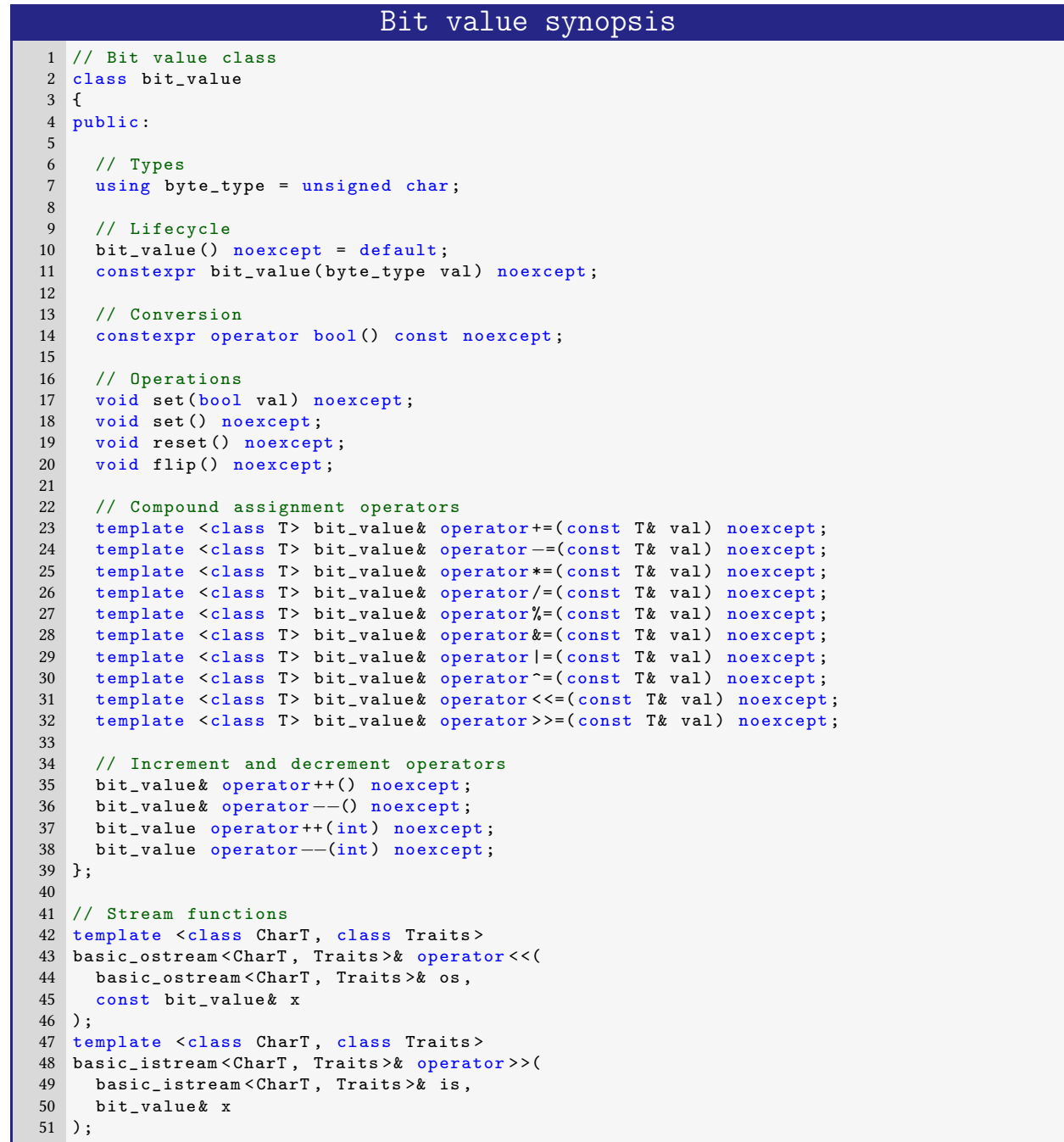


Figure 6: Bit value technical specifications

Bit reference specifications

The specifications of `std::bit_reference` are given on figures 7 and 8.

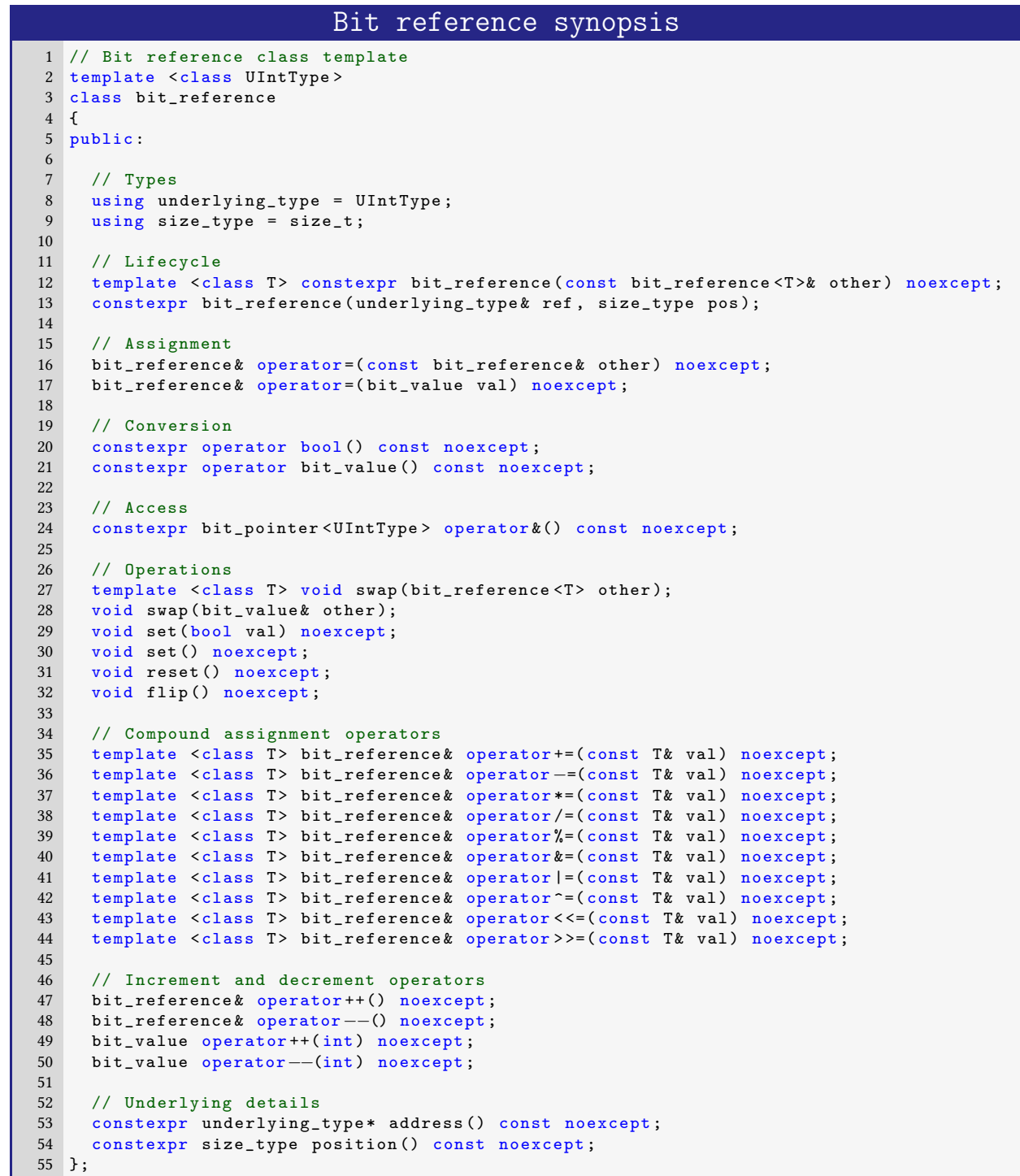


Figure 7: Bit reference technical specifications

Bit reference non-member functions

```
1 // Swap and exchange
2 template <class T, class U>
3 void swap(
4     bit_reference<T> lhs,
5     bit_reference<U> rhs
6 ) noexcept;
7 template <class T>
8 void swap(
9     bit_reference<T> lhs,
10    bit_value& rhs
11 ) noexcept;
12 template <class T>
13 void swap(
14     bit_value& lhs,
15     bit_reference<T> rhs
16 ) noexcept;
17 template <class T, class U = bit_value>
18 bit_value exchange(
19     bit_reference<T> x,
20     U&& val
21 );
22
23 // Stream functions
24 template <class CharT, class Traits, class T>
25 basic_ostream<CharT, Traits>& operator<< (
26     basic_ostream<CharT, Traits>& os,
27     const bit_reference<T>& x
28 );
29 template <class CharT, class Traits, class T>
30 basic_istream<CharT, Traits>& operator>> (
31     basic_istream<CharT, Traits>& is,
32     const bit_reference<T>& x
33 );
34
35 // Make function
36 template <class T>
37 constexpr bit_reference<T> make_bit_reference(
38     T& ref,
39     typename bit_reference<T>::size_type pos
40 );
```

Figure 8: Bit reference non-member functions

Bit pointer specifications

The specifications of `std::bit_pointer` are given on figures 9 and 10.

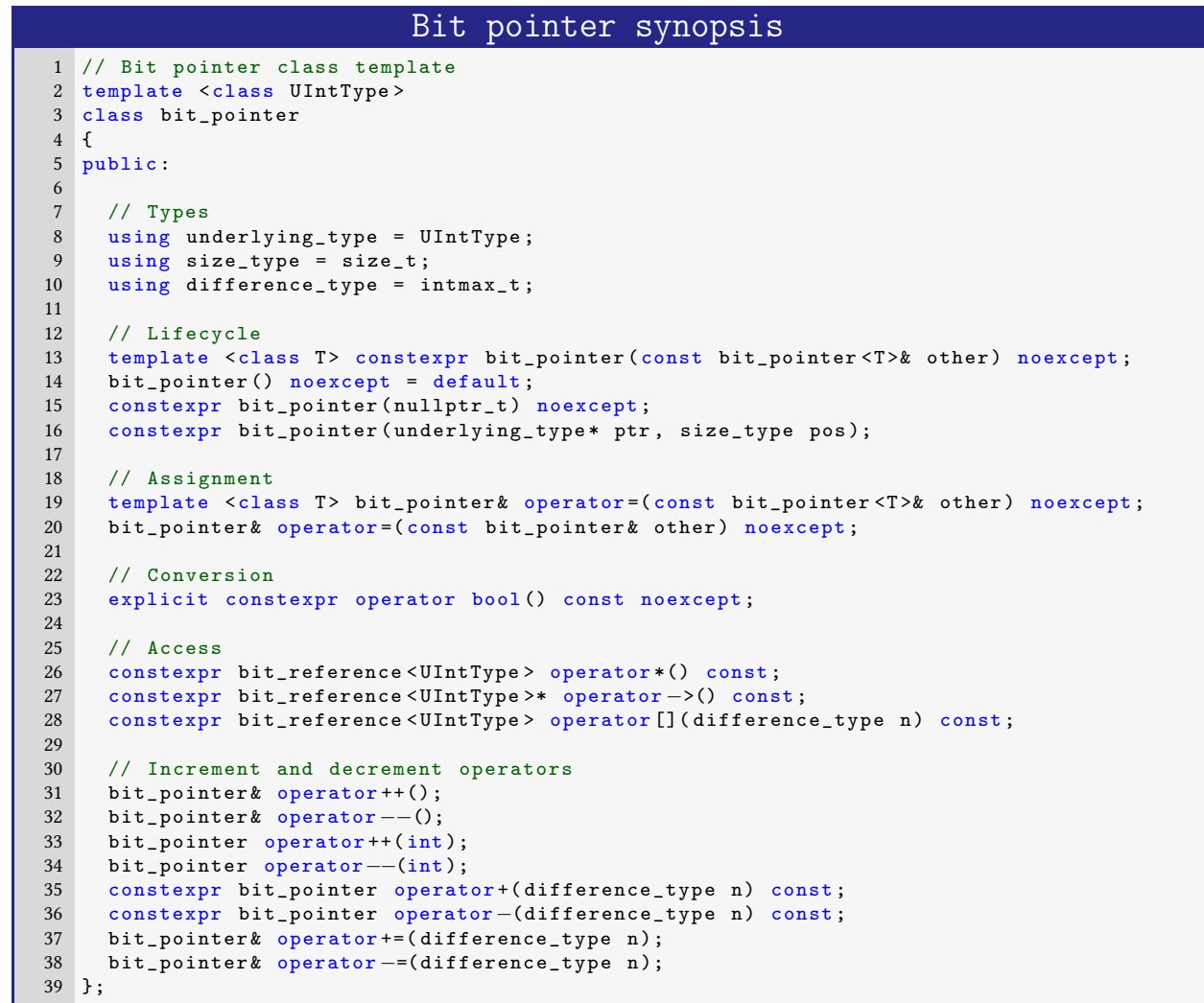


Figure 9: Bit pointer technical specifications

Bit pointer non-member functions

```
1 // Non-member arithmetic operators
2 template <class T>
3 constexpr bit_pointer<T> operator+(
4     typename bit_pointer<T>::difference_type n,
5     const bit_pointer<T>& x
6 );
7 template <class T, class U>
8 typename common_type<
9     typename bit_pointer<T>::difference_type,
10    typename bit_pointer<U>::difference_type
11 >::type operator-(
12     const bit_pointer<T>& lhs,
13     const bit_pointer<U>& rhs
14 ) noexcept;
15
16 // Comparison operators
17 template <class T, class U>
18 constexpr bool operator==(
19     const bit_pointer<T>& lhs,
20     const bit_pointer<U>& rhs
21 ) noexcept;
22 template <class T, class U>
23 constexpr bool operator!=(
24     const bit_pointer<T>& lhs,
25     const bit_pointer<U>& rhs
26 ) noexcept;
27 template <class T, class U>
28 constexpr bool operator<(
29     const bit_pointer<T>& lhs,
30     const bit_pointer<U>& rhs
31 ) noexcept;
32 template <class T, class U>
33 constexpr bool operator<=(
34     const bit_pointer<T>& lhs,
35     const bit_pointer<U>& rhs
36 ) noexcept;
37 template <class T, class U>
38 constexpr bool operator>(
39     const bit_pointer<T>& lhs,
40     const bit_pointer<U>& rhs
41 ) noexcept;
42 template <class T, class U>
43 constexpr bool operator>=(
44     const bit_pointer<T>& lhs,
45     const bit_pointer<U>& rhs
46 ) noexcept;
47
48 // Make function
49 template <class T>
50 constexpr bit_pointer<T> make_bit_pointer(
51     T* ptr,
52     typename bit_pointer<T>::size_type pos
53 );
```

Figure 10: Bit pointer non-member functions

Bit iterator specifications

The specifications of `std::bit_iterator` are given on figures 11 and 12.



Figure 11: Bit iterator technical specifications

Bit iterator: non-member functions

```
1 // Non-member arithmetic operators
2 template <class T>
3 bit_iterator<T> operator+(
4     typename bit_iterator<T>::difference_type n,
5     const bit_iterator<T>& i
6 );
7 template <class T, class U>
8 typename common_type<
9     typename bit_iterator<T>::difference_type,
10    typename bit_iterator<U>::difference_type
11 >::type operator-(
12     const bit_iterator<T>& lhs,
13     const bit_iterator<U>& rhs
14 );
15
16 // Comparison operators
17 template <class T, class U>
18 bool operator==(
19     const bit_iterator<T>& lhs,
20     const bit_iterator<U>& rhs
21 );
22 template <class T, class U>
23 bool operator!=(
24     const bit_iterator<T>& lhs,
25     const bit_iterator<U>& rhs
26 );
27 template <class T, class U>
28 bool operator<(
29     const bit_iterator<T>& lhs,
30     const bit_iterator<U>& rhs
31 );
32 template <class T, class U>
33 bool operator<=(
34     const bit_iterator<T>& lhs,
35     const bit_iterator<U>& rhs
36 );
37 template <class T, class U>
38 bool operator>(
39     const bit_iterator<T>& lhs,
40     const bit_iterator<U>& rhs
41 );
42 template <class T, class U>
43 bool operator>=(
44     const bit_iterator<T>& lhs,
45     const bit_iterator<U>& rhs
46 );
47
48 // Make function
49 template <class T>
50 bit_iterator<T> make_bit_iterator(
51     const T& i,
52     typename bit_iterator<T>::size_type pos
53 );
```

Figure 12: Bit iterator non-member functions

6 Alternative technical specifications

Introduction

Accordingly to the feedback gathered online, we decided to detail an alternative design. This suggestion of design is based on the alternative that consists in considering that the mathematical definition of a bit is a pure digit, and only a digit, and, as is, it should not provide any arithmetic behaviour. This can be easily achieved by stripping `std::bit_value` and `std::bit_reference` from their operators and from their implicit conversion members, and by keeping `std::bit_pointer` and `std::bit_iterator` the same.

Alternative bit value specifications

The alternative specifications of `std::bit_value` are given on figure 13.

Alternative bit reference specifications

The alternative specifications of `std::bit_reference` are given on figure 14.

Bit pointer specifications

The specifications of `std::bit_pointer` are given on figures 9 and 10.

Bit iterator specifications

The specifications of `std::bit_iterator` are given on figures 11 and 12.

Alternative bit value synopsis

```
1 // Alternative bit value class
2 class bit_value
3 {
4 public:
5
6     // Types
7     using byte_type = unsigned char;
8
9     // Lifecycle
10    bit_value() noexcept = default;
11    explicit constexpr bit_value(byte_type val) noexcept;
12
13    // Assignment
14    bit_value& operator=(byte_type val) noexcept;
15
16    // Conversion
17    explicit constexpr operator bool() const noexcept;
18
19    // Operations
20    void set(bool val) noexcept;
21    void set() noexcept;
22    void reset() noexcept;
23    void flip() noexcept;
24 };
25
26 // Comparison operators
27 constexpr bool operator==(
28     const bit_value& lhs,
29     const bit_value& rhs
30 ) noexcept;
31 constexpr bool operator!=(
32     const bit_value& lhs,
33     const bit_value& rhs
34 ) noexcept;
35 constexpr bool operator<(
36     const bit_value& lhs,
37     const bit_value& rhs
38 ) noexcept;
39 constexpr bool operator<=(
40     const bit_value& lhs,
41     const bit_value& rhs
42 ) noexcept;
43 constexpr bool operator>(
44     const bit_value& lhs,
45     const bit_value& rhs
46 ) noexcept;
47 constexpr bool operator>=(
48     const bit_value& lhs,
49     const bit_value& rhs
50 ) noexcept;
51
52 // Stream functions
53 template <class CharT, class Traits>
54 basic_ostream<CharT, Traits>& operator<<(
55     basic_ostream<CharT, Traits>& os,
56     const bit_value& x
57 );
58 template <class CharT, class Traits>
59 basic_istream<CharT, Traits>& operator>>(
60     basic_istream<CharT, Traits>& is,
61     bit_value& x
62 );
```

Figure 13: Alternative bit value technical specifications

Alternative bit reference synopsis

```

1 // Alternative bit reference class template
2 template <class UIntType>
3 class bit_reference
4 {
5 public:
6
7     // Types
8     using underlying_type = UIntType;
9     using size_type = size_t;
10
11     // Lifecycle
12     template <class T> constexpr bit_reference(const bit_reference<T>& other) noexcept;
13     constexpr bit_reference(underlying_type& ref, size_type pos);
14
15     // Assignment
16     bit_reference& operator=(const bit_reference& other) noexcept;
17     bit_reference& operator=(bit_value val) noexcept;
18
19     // Conversion
20     constexpr operator bit_value() const noexcept;
21
22     // Access
23     constexpr bit_pointer<UIntType> operator&() const noexcept;
24
25     // Operations
26     template <class T> void swap(bit_reference<T> other);
27     void swap(bit_value& other);
28     void set(bool val) noexcept;
29     void set() noexcept;
30     void reset() noexcept;
31     void flip() noexcept;
32
33     // Underlying details
34     constexpr underlying_type* address() const noexcept;
35     constexpr size_type position() const noexcept;
36 };
37
38 // Swap and exchange
39 template <class T, class U>
40 void swap(
41     bit_reference<T> lhs,
42     bit_reference<U> rhs
43 ) noexcept;
44 template <class T>
45 void swap(
46     bit_reference<T> lhs,
47     bit_value& rhs
48 ) noexcept;
49 template <class T>
50 void swap(
51     bit_value& lhs,
52     bit_reference<T> rhs
53 ) noexcept;
54 template <class T, class U = bit_value>
55 bit_value exchange(
56     bit_reference<T> x,
57     U&& val
58 );
59
60 // Make function
61 template <class T>
62 constexpr bit_reference<T> make_bit_reference(
63     T& ref,
64     typename bit_reference<T>::size_type pos
65 );

```

Figure 14: Alternative bit reference technical specifications

7 Discussion and open questions

As a first version, the intent of this proposal is to start a discussion about the introduction of basic bit utilities in the standard library. Several design options have been detailed in section 4, and the specification presented in part 5 represents only one option amongst multiple alternatives. Answering the following questions are of primary importance regarding design and specification choices:

- What types should be allowed as template parameters of `std::bit_reference` and `std::bit_pointer`? Only unsigned integers? All integral types? And what about bit containers?
- What functionalities and arithmetic should a bit implement? A design with `set`, `reset` and `flip` operators? Or one emulating a `bool` and nothing else? Or one adding the arithmetic behaviour of an unsigned integer of exactly one bit?
- Should `std::bit_value` be introduced to improve the global design? Should the naming `std::bit` be used instead of `std::bit_value`, even though the class is not a bit and only mimics the behaviour of a non-referenced bit value?
- Should bit references be both implicitly convertible to `bool` and `std::bit_value`?
- Should bit pointers be implicitly or explicitly convertible to `bool` to check their state?
- Should other overloads of `std::swap` and `std::exchange` be provided as described in the [additional design remarks](#) subsection?
- Should `const` versions of the class templates be provided separately in order to replace the solution consisting in passing `const T` as template parameters? Or should typedefs referring to `std::bit_reference<const T>` and `std::bit_pointer<const T>` be provided?
- How should the internal details, namely the address of the underlying value and the bit position, be accessed? Are `underlying_type`, `address` and `position` good names for these underlying details? Should the `pos` parameter be of type `std::size_t`?
- Should `set`, `reset` and `flip` be provided as non-member functions with different names to avoid conflict with `std::set`, even though these functionalities are very particular to bits?
- What should happen when `pos >= std::numeric_limits<T>::digits`?
- Should a type traits helper structure such as `std::iterable_bits` be introduced to count the number of iterable bits of unsigned integral types in order to replace `std::numeric_limits<T>::digits`?
- What functions should be specified as `constexpr` and what members should be specified as `noexcept`? In particular, could the constructors of `std::bit_iterator` and its base member function be marked as `constexpr` to facilitate compile-time computation?
- Should any relation be introduced between `std::bit_reference` and `std::bitset::reference`? Or should they be kept as two completely independent entities in terms of design as in this version of the proposal?
- Would this design be compatible with the range proposal and a future range of bits? In this regard, should `std::bit_value` be parameterized by a template type?

Answering and achieving a consensus on these questions should lead to a minimalist but very versatile set of tools to manipulate unique bits. We have chosen to illustrate the two approaches that we consider to be the most consistent: either a bit with the values 0 or 1 can be considered as a number, and in that case, one of the best option is to provide the arithmetic behaviour of a one-bit long unsigned integer, or it should be considered as a pure digit and therefore have no arithmetic operators. Between these two options, there is a grey area, that we find to be very error-prone. Identifying bits and boolean values is one of them, since the arithmetic of `bool` implicitly promoted to `int` is particularly non-intuitive as illustrated in the introductory listing of section 4. A bit and a `bool` are conceptually two different objects, or, in other words, a bit is *not* a `bool`. Even in the current standard, `std::bitset::reference` and `std::vector<bool>::reference` do not mimic booleans: a `bool` has compound assignment operators, that the two classes do not have, and the behaviour of the `operator~` is very different for a `bool` and for the nested classes. We argue that one of the main fundamental reason why the template specialization `std::vector<bool>` is considered by many as a bad design decision, can be boiled down to the fact that bits and booleans are two different things, even if both happen to have two values. The same decision was not made for `std::array`: an array of `bool` and an array of bits are two different things, and the last one is named a `std::bitset`. Both the design we illustrated, include `std::bit_value`: the first one requires it for arithmetic operations, and the second one requires it to block all implicit conversions to `bool` that would lead to confusion.

Bit manipulation algorithms should be the subject of another proposal built on the top of the fundamental layer discussed here. Such a library could include a `std::bit_view`, as well as specializations of the standard algorithms. As already mentioned in section 2, thanks to the `address` and `position` members, the algorithms could operate on the `underlying_type` instead of operating on each bit, thus providing a significant speedup. For example, `std::count` could call the `popcnt` assembly function when operating on bit iterators. Moreover the set of standard algorithms could be extended with algorithms dedicated to bit operations. These extensions could include, amongst others, algorithms inspired by the very exhaustive proposal N3864 [Fioravante, 2014], algorithms implementing unsigned unbounded integer arithmetic, and algorithms based on the [Bit Manipulation Instruction sets](#) such as `parallel_bit_deposit` and `parallel_bit_extract`.

The resulting bit library could serve a wide range of purposes, from cryptography to video games, and from arbitrary-precision integral arithmetic to high performance computing. And, of course, it could finally offer a proper way to use unsigned integers as bit containers.

8 Acknowledgements

The authors would like to thank Howard Hinnant, Jens Maurer, Tony Van Eerd, Klemens Morgenstern, Vicente Botet Escriba, Tomasz Kaminski, Odin Holmes and the other contributors of the [ISO C++ Standard - Discussion](#) and of the [ISO C++ Standard - Future Proposals](#) groups for their initial reviews and comments.

Vincent Reverdý and Robert J. Brunner have been supported by the National Science Foundation Grant AST-1313415. Robert J. Brunner has been supported in part by the Center for Advanced

9 References

- [ISO, 2014] (2014). Information technology – programming languages – c++. Technical Report 14882:2014, ISO/IEC.
- [Becker, 2014] Becker, P. (2014). Proposal for unbounded-precision integer types. Technical Report N4038, ISO/IEC JTC1/SC22/WG21 - The C++ Standards Committee.
- [Carruth, 2014] Carruth, C. (2014). Efficiency with algorithms, performance with data structures. <https://youtu.be/fHNmRkzxHws>.
- [Fioravante, 2014] Fioravante, M. (2014). A constexpr bitwise operations library for c++. Technical Report N3864, ISO/IEC JTC1/SC22/WG21 - The C++ Standards Committee.
- [Hinnant, 2012] Hinnant, H. (2012). On vector<bool>. <https://isocpp.org/blog/2012/11/on-vectorbool>.
- [Jones, 2011] Jones, D. (2011). Working draft, standard for programming language c. Technical Report N1570, ISO/IEC JTC1/SC22/WG14 - The C Standards Committee.
- [Ozturk et al., 2012] Ozturk, E., Guilford, J., Gopal, V., and Feghali, W. (2012). New instructions supporting large integer arithmetic on intel architecture processors. Technical report, Intel.
- [Reverdy, 2016] Reverdy, V. (2016). Implementation of bit utility class templates. https://github.com/vreverdy/bit_utilities.
- [Siek et al., 2015] Siek, J. et al. (2015). Boost dynamic bitset. http://www.boost.org/doc/libs/1_59_0/libs/dynamic_bitset/dynamic_bitset.html.
- [Smith, 2015] Smith, R. (2015). Working draft, standard for programming language c++. Technical Report N4567, ISO/IEC JTC1/SC22/WG21 - The C++ Standards Committee.
- [Stroustrup, 2013] Stroustrup, B. (2013). *The C++ Programming Language*. Addison-Wesley Professional, 4th edition.
- [Yangtao, 2013] Yangtao, C. (2013). How to use bit-band and bme on the ke04 and ke06 subfamilies. Technical Report AN4838, Freescale Semiconductor.